

Text as Data Exam Crib Sheet

Regular Expressions

.	any character
[abc]	one of a, b, c
[^abc]	any char except a, b, c
[a-z]	range
\d, \w, \s	digit, word char, whitespace
\D, \W, \S	negated versions
*/+/?	0 or more / 1 or more / 0 or 1
^/\$	start/end of string
\b	word boundary

1. Tokens & Similarity

NLP pipeline steps:

- Data cleaning: Removing non-relevant links, tables, etc from documents
- Tokenization: split text into tokens (possibly sentence splitting too)
- Normalization: stemming vs lemmatization.
- Stopword removal.

Similarity measures:

- Number of overlapping words: $\text{sim}(X, Y) = |X \cap Y|$
- Overlap Coefficient: $\frac{|X \cap Y|}{\min(|X|, |Y|)}$
- Sørensen–Dice Coefficient: $\frac{2|X \cap Y|}{|X| + |Y|}$
- Jaccard Similarity: $\frac{|X \cap Y|}{|X \cup Y|}$
- $\text{similarity}(X, Y) \propto -\text{distance}(X, Y)$

Properties of a metric:

- 1. Identity: $d(X, X) = 0$
- 2. Positivity: $d(X, Y) > 0$ where $X \neq Y$
- 3. Symmetry: $d(X, Y) = d(Y, X)$
- 4. Triangle Inequality: $d(X, Z) \leq d(X, Y) + d(Y, Z)$

encodings: ASCII (1 byte) / Unicode (2-4 bytes)

2. Vectors & Clustering

- TF: term frequency in document
- DF: document count containing term
- IDF: inverse DF as fewer docs is more meaningful
- TF-IDF: $tf \times idf$.
- Zipf: few words frequent, many rare.

Comparing vectors:

- Euclidean: $d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$
- Cosine: $\cos(\theta) = \frac{\vec{d}_1 \cdot \vec{d}_2}{\|\vec{d}_1\| \|\vec{d}_2\|}$.

Equations:

$$\text{log-transformed df: } \begin{cases} 1 + \log_{10} \text{tf}_{t,d}, & \text{if } \text{tf}_{t,d} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$$\text{log-transformed idf: } \log\left(\frac{N}{\text{df}_t}\right)$$

$$\text{tf-idf: } (1 + \log(\text{tf}_{t,d})) \cdot \log\left(\frac{N}{\text{df}_t}\right)$$

K-Means Clustering

- Step 0: Initialize centroids
- Step 1: Assign each data point to its nearest centroid
- Step 2: Recalculate the centroids (as the average of assigned points)
- Then repeat Steps 1 and 2 until convergence

Centroid Initialization Options

- Randomly select k data points from data set
- Forgy partition - Randomly select k data points from the dataset and use them as your initial centroids

- Random Partition - Assign each point randomly to a cluster and calculate the centroids from each cluster

Elbow method for picking k

Measuring Clustering

- Look at data (smell test, oddities, etc)
- Intrinsic: (judging the clustering directly)
 - Cohesion score: average distance of i to the objects in the same cluster
 - Separation score: min (average distance of i to objects in another cluster)
 - Human evaluation or checking purity (requiring external labels)
- Extrinsic: Clustering as part of another system that can be evaluated (e.g. classification)

$\text{silhouette}(i) =$

$$\begin{cases} 1 - \frac{\text{cohesion}(i)}{\text{separation}(i)}, & \text{if } \text{cohesion}(i) < \text{separation}(i), \\ 0, & \text{if } \text{cohesion}(i) = \text{separation}(i), \\ \frac{\text{separation}(i)}{\text{cohesion}(i)} - 1, & \text{if } \text{cohesion}(i) > \text{separation}(i). \end{cases}$$

3. Language Modelling

Probability:

- $0 \leq P(x) \leq 1$ and $\sum_x P(x) = 1$
- $P(A | B) = \frac{P(B|A) \cdot P(A)}{P(B)}$
- Chain rule: $P(A, B, C) = P(A|B, C)P(B|C)P(C)$.
- A and B are independent if
 - $P(A | B) = P(A)$ and $P(B | A) = P(B)$

Language Models:

- Goal: $P(w_1, \dots, w_n) / P(w_{k+1} | w_1, \dots, w_k)$.
- Counting: $P(w_1, w_2, \dots, w_n) = \frac{\text{Count}(w_1, w_2, \dots, w_n)}{N}$
- Markov assumption and n-grams
- Add-k smoothing: $\frac{\text{count} + k}{N + kV}$.
- $P_{\text{add-k}}(t | \theta) = \frac{\text{count}(t, \theta) + k}{\sum_{t'} \text{count}(t', \theta) + k|V|}$
- $P_{\text{interpolated}}(t_i | t_{i-1}, t_{i-2}) = \lambda_1 P(t_i | t_{i-1}, t_{i-2}) + \lambda_2 P(t_i | t_{i-1}) + \lambda_3 P(t_i)$
- $P_{\text{Jelinek-mercer}}(t | \theta_d, \theta_c) = \lambda P(t | \theta_d) + (1 - \lambda) P(t | \theta_c)$

LM Evaluation:

- Syntactically – match “rules” of language
- Semantically – conveys actual meaning
- Extrinsic (as part of another task) or Intrinsic (evaluate LM task)
- Perplexity: $(\prod P(w_i))^{-1/n} = 2^{\text{Cross-Entropy}}$
- Cross-entropy: $-\frac{1}{n} \sum \log_2 P(w_i)$
- Entropy: $-\sum P(x) \log_2 P(x)$.

Generation:

- Use LM to generate probabilities for next token
- Pick the token from vocab with highest prob (boring)
- Sample from the probability distribution
- Beam search: Grow several candidate short seqs (token-by-token) and discard unlikely ones
- Other ideas: top-k, top-p, contrastive search

4. Text Classification

- Binary, multi-class, multi-label.
- Supervised (labelled data), unsupervised (no labels)
- Metrics:
 - Accuracy: $\frac{TP+TN}{TP+FP+TN+FN}$.

- Precision: $\frac{TP}{TP+FP}$.
- Recall: $\frac{TP}{TP+FN}$.
- F1: $\frac{2 \times TP}{2 \times TP + FP + FN} = 2 \cdot \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$.
- $F_\beta = (1 + \beta^2) \frac{\text{precision} \times \text{recall}}{\beta^2 \times \text{precision} + \text{recall}}$
- Metrics for Multi-class problems:
 - Accuracy = correct/total
 - Macro: unweighted average of metric
 - Weighted: weighted average of metric based on sample count
 - Micro: Calculate TP/FP/TN/FN for each class, add up and then calculate precision, etc
- ROC curve
 - Area under (AUROC): 0.5 is random
 - Start bottom left (and at start of ordered predictions). Actual Positive: move up. Actual Negative: move right
- Hyperparameter tuning
- Training/validation/test splits
- K-Fold Cross-validation (for small datasets)

Classifier problems

- Noise (labels/features) in the data
- Needed features are just not there
- Underfitting vs Overfitting
- Class imbalance

5. Context Word Embeddings

- Static: IBM model and SVD
 - Handles synonymy – when two words have similar/identical meanings.
 - Don't handle polysemy – when one word has multiple meanings.
- Contextual: ELMo, BERT, GPT.

Self-Attention:

- Q = Input vectors multiplied by W_Q = queries
- K = Input vectors multiplied by W_K = keys
- V = Input vectors multiplied by W_V = values
- Attention: $Att(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$.

Subword-Tokenization:

- BPE algorithm
 - Pretokenize into words
 - Create a vocabulary of symbols
 - Repeat until the desired vocab size is reached
 - * Find most common neighbouring symbols
 - * Replace all instances with a new character and add to vocab
- Special Tokens: [CLS], [SEP], [PAD], [MASK].

Transformers:

- Multi-head attention, positional encoding.
- Causal LM task: Predict the next word
- Masked LM task: Predict a masked word
- Other tasks: Does one sentence follow the other? (BERT) and Spot the corrupted word
- Encoder (e.g. BERT) versus decoder (e.g. GPT)
- Pretraining + fine-tuning (transfer learning)

6. POS Tagging & Parsing

- Transitions: $P(y_i|y_{i-1})$ and Emissions: $P(x_i|y_i)$.

- Greedy: take best transition/emission at each token
- Viterbi: Optimal path through transitions/emissions
- Constituency parsing: NPs, VPs and PPs
- Dependency Parsing: directed graph, weakly connected, acyclic
- Transition-based: Shift, Left-Arc, Right-Arc.

7. Ethics

- Bias: models inherit data bias
- Environmental cost: huge compute
- Transparency: black-box models
- Copyright, liability, education impact
- Best practices: datasheets, model cards

8. Information Extraction

- NER: IOB tagging with CRF / transformers
- Entity Linking: from strings to things
 - candidates with dict method / dense vector retrieval
 - (optional) rerank with transformer model
- Relation Extraction: triples (subj, rel, obj)
 - co-occurrence counts (no relation)
 - rule-based (e.g. "X, who starred in Y")
 - BERT-based with pooling and entity markers
- Coreference + anaphora/cataphora/exophora
 - mention detection (pronouns, entities, NPs)
 - Hobbs: Finds noun-phrase that a pronoun refers to
 - Clustering mentions (e.g. decide two mentions refer to same thing)

9. Large Language Models

- Training
 - Instruction tuning
 - Reinforcement learning from human feedback (for alignment)
- Prompt engineering: zero-shot, few-shot, explicit, examples, CoT.
- Issues: factuality, dangerous generations, test set leakage, expense

String Operations

- $a + b$ - Concatenate strings a and b
- `text[i:j]` - Get substring from position i to j
- `text[:i]` - Get final i characters
- `.upper()` / `.lower()` - Uppercase and lowercase text
- `.isupper()` / `.islower()` - Check if upper/lowercase
- `.lstrip()` / `.rstrip()` / `.strip()` - Remove whitespace from (left,right,both) side(s)
- `.replace(a,b)` - Replace substring a with b
- `.split(a)` - Split string by a (default = whitespace)
- `.removeprefix(...)` / `.removesuffix(...)` - Remove prefix/suffix
- `.startswith(...)` / `.endswith(...)` - Check if string starts/ends with substring
- `.find(...)` / `.count(...)` - Find/count substrings
- `a.join(l)` - Join list l with delimiter a